HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO BÀI TẬP NHÓM

MÔN: CÁC HỆ THỐNG PHÂN TÁN

ĐỀ TÀI DS02

XÂY DỰNG HỆ THỐNG CHIA SẺ FILE NGANG HÀNG MÔ PHỎNG GIAO THỨC BITTORRENT

Giảng viên hướng dẫn: **Kim Ngọc Bách**

Nhóm thực hiện: **Nhóm 8**

Tạ Quang Vinh	B23DCDT289
Ngô Minh Tùng	B23DCDT279
Lương Sơn Tùng	B23DCVT448

Hà Nội – 2026

LỜI NÓI ĐẦU

Trong môn Các hệ thống phân tán, nhóm em được giao đề tài DS02: xây dựng một hệ thống chia sẻ file ngang hàng (P2P) theo kiểu BitTorrent. Đây là đề tài khá "đúng chất" phân tán: không có một máy chủ nào giữ toàn bộ dữ liệu, mỗi máy vừa là người tải vừa là người phục vụ, các máy có thể vào ra hoặc sập bất cứ lúc nào, và hệ thống vẫn phải đảm bảo file tải về đúng từng byte.

Nhóm chọn Python để cài đặt vì viết nhanh, dễ đọc và có sẵn thư viện cho socket, HTTP và đa luồng. Ngoài phần lõi (tracker và peer), nhóm còn làm thêm bộ mô phỏng để bật hàng chục tiến trình peer, gây churn (peer vào/ra/sập), làm sập tracker, rồi đo đạc và vẽ biểu đồ. Nhờ vậy nhóm không chỉ "chạy được" mà còn đánh giá được hệ thống trong nhiều tình huống khác nhau.

Báo cáo gồm 6 chương: giới thiệu đề tài, cơ sở lý thuyết, thiết kế hệ thống, cài đặt từng module, kiểm thử và thực nghiệm, cuối cùng là kết luận. Do thời gian và kiến thức còn hạn chế nên báo cáo khó tránh khỏi thiếu sót, nhóm em rất mong nhận được góp ý của thầy để hoàn thiện hơn.

Nhóm em xin chân thành cảm ơn thầy **Kim Ngọc Bách** đã hướng dẫn và giảng dạy môn học trong suốt học kỳ.

Hà Nội, tháng 11 năm 2026

Nhóm 8

MỤC LỤC

Lời nói đầu **ii**

Danh mục từ viết tắt **vi**

1 Giới thiệu đề tài **1**

1.1 Đặt vấn đề 1

1.2 Mục tiêu của đề tài 1

1.3 Phạm vi 1

1.4 Phân công công việc 2

2 Cơ sở lý thuyết **3**

2.1 Mô hình mạng ngang hàng (P2P) 3

2.2 Giao thức BitTorrent 3

2.2.1 Các thành phần 3

2.2.2 Chunk và kiểm tra tính toàn vẹn 3

2.2.3 Chiến lược chọn chunk: rarest-first và endgame 4

2.2.4 Choke / unchoke 4

2.3 Churn và các mô hình xác suất 4

2.4 Các khái niệm hệ phân tán liên quan 5

3 Thiết kế hệ thống **6**

3.1 Kiến trúc tổng thể 6

3.2 Năm hợp đồng giao tiếp 6

3.2.1 Hợp đồng 1 – Metainfo 7

3.2.2 Hợp đồng 2 – API tracker 7

3.2.3 Hợp đồng 3 – Wire protocol 8

3.2.4 Hợp đồng 4 – CLI của peer 8

3.2.5 Hợp đồng 5 – Log sự kiện và kịch bản 9

3.3 Luồng hoạt động của một leecher 10

4 Cài đặt hệ thống **11**

4.1 Module A – Tracker và tầng mạng . . . 11

4.1.1 Máy chủ tracker (A1–A3) . . . 11

4.1.2 Wire protocol (A4) . . . 12

4.1.3 Tầng kết nối TCP (A5) . . . 12

4.1.4 Client tracker có backoff (A6) . . . 13

4.2 Module B – Peer core . . . 13

4.2.1 Metainfo và lưu trữ chunk (B1, B2) . . . 13

4.2.2 Chọn chunk: rarest-first với endgame (B3) . . . 13

4.2.3 Tải song song (B4) . . . 14

4.2.4 Phục vụ chunk – upload (B5) . . . 14

4.2.5 Chịu lỗi phía peer (B6) . . . 15

4.2.6 Tiến trình peer hoàn chỉnh (B7) . . . 15

4.3 Module C – Mô phỏng và đánh giá . . . 15

4.3.1 Bộ khởi chạy kịch bản (C1) . . . 15

4.3.2 Engine churn (C2) . . . 16

4.3.3 Giả lập mạng (C3) . . . 16

4.3.4 Thu số liệu, kiểm thử và demo (C4, C5, C7) . . . 16

5 Kiểm thử và thực nghiệm **17**

5.1 Môi trường thực nghiệm . . . 17

5.2 Kiểm thử . . . 17

5.3 Các kịch bản thí nghiệm . . . 18

5.4 Kết quả . . . 18

5.4.1 Baseline . . . 19

5.4.2 Multi-seed . . . 20

5.4.3 High churn . . . 20

5.4.4 Flash crowd . . . 20

5.4.5 Tracker crash . . . 21

5.5 Đo tải tracker . . . 23

5.6 Demo . . . 23

6 Kết luận và hướng phát triển **25**

6.1 Kết quả đạt được 25

6.2 Hạn chế 25

6.3 Hướng phát triển 26

Tài liệu tham khảo **27**

DANH MỤC HÌNH VẼ

3.1 Kiến trúc tổng thể của hệ thống (mỗi cặp peer đều có thể nối TCP trực tiếp) . . 6
3.2 Khung thông điệp của wire protocol 8
3.3 Các bước chính của một leecher 10

5.1 Thời gian tải của các leecher theo từng kịch bản 19
5.2 Tỉ lệ leecher tải xong theo từng kịch bản 19
5.3 Hệ số nhân bản trung bình của chunk theo thời gian 20
5.4 Số announce tới tracker mỗi giây (đánh dấu khoảng tracker sập) 21
5.5 Thông lượng tải xuống / tải lên của swarm theo thời gian 22
5.6 Tỉ lệ dữ liệu do seed và do leecher phục vụ 23

DANH MỤC BẢNG BIỂU

1.1 Phân công nhiệm vụ trong nhóm 2

2.1 Liên hệ giữa đề tài và các khái niệm của môn học 5

3.1 Các hợp đồng giao tiếp giữa các module 7
3.2 Các loại thông điệp giữa hai peer 8

5.1 Các nhóm kiểm thử 17
5.2 Các kịch bản thí nghiệm 18
5.3 Tổng hợp kết quả thí nghiệm 18
5.4 Kết quả đo tải tracker 23

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
P2P	Peer-to-Peer – mạng ngang hàng
TCP	Transmission Control Protocol
HTTP	HyperText Transfer Protocol
SHA-1, SHA-256	Secure Hash Algorithm (160 bit, 256 bit)
TTL	Time To Live – thời gian sống của một bản ghi
CLI	Command Line Interface – giao diện dòng lệnh
JSONL	JSON Lines – mỗi dòng là một đối tượng JSON
YAML	YAML Ain't Markup Language – định dạng file cấu hình
E2E	End-to-End – kiểm thử toàn hệ thống
seed	Peer đã có đủ file, chỉ phục vụ
leecher	Peer đang tải, còn thiếu chunk
churn	Hiện tượng peer liên tục tham gia / rời / sập khỏi mạng

CHƯƠNG 1

GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Với mô hình client–server truyền thống, khi nhiều người cùng tải một file lớn thì máy chủ phải gánh toàn bộ băng thông: càng đông người tải thì mỗi người càng chậm, và nếu máy chủ sập thì không ai tải được nữa. Mô hình ngang hàng (P2P) giải quyết vấn đề này bằng cách để chính những người đang tải chia sẻ lại phần dữ liệu họ đã có. Số người tải càng nhiều thì tổng băng thông phục vụ càng lớn.

BitTorrent là giao thức P2P phổ biến nhất theo hướng này. File được chia thành nhiều *chunk* (piece) nhỏ, mỗi chunk có mã băm riêng; một *tracker* giúp các peer tìm thấy nhau; sau đó các peer trao đổi chunk trực tiếp với nhau và tự kiểm tra tính đúng của từng chunk.

1.2 Mục tiêu của đề tài

Đề tài DS02 yêu cầu nhóm xây dựng một hệ thống chia sẻ file P2P mô phỏng BitTorrent, cụ thể:

- Chia file thành các chunk, mỗi chunk có mã băm SHA-1, mô tả trong một file metainfo.
- Xây dựng **tracker** cho phép peer đăng ký và lấy danh sách peer khác trong cùng swarm.
- Xây dựng **peer** vừa tải vừa phục vụ chunk, tải song song từ nhiều nguồn, chọn chunk hiếm trước (rarest-first).
- Chịu lỗi: peer nguồn chết giữa chừng thì chuyển sang nguồn khác; chunk sai hash bị loại; tracker sập rồi bật lại thì swarm tự hồi phục.
- Mô phỏng **churn** (peer vào/ra/sập theo phân phối xác suất), đo đạc và đánh giá hiệu năng.
- Có bộ kiểm thử tự động và một bản demo chạy bằng một lệnh.

1.3 Phạm vi

Hệ thống chạy trên một máy (địa chỉ `127.0.0.1`), mỗi tracker/peer là một **tiến trình hệ điều hành riêng**, giao tiếp qua TCP và HTTP thật. Nhóm không làm các phần nâng cao của BitTorrent

như DHT, magnet link, mã hóa kết nối hay NAT traversal. Định dạng metainfo dùng JSON thay cho bencode để dễ đọc và debug.

1.4 Phân công công việc

Nhóm chia hệ thống thành ba module, mỗi người sở hữu một module và chỉ "gặp nhau" ở các hợp đồng giao tiếp đã chốt từ đầu. Mỗi người cũng tự viết bản giả (mock) của phần còn lại để không phải chờ nhau.

Bảng 1.1: Phân công nhiệm vụ trong nhóm

Thành viên	**Module**	**Đầu việc chính**
Lương Sơn Tùng B23DCVT448	A – Tracker & tầng mạng	A1–A7: tracker (announce, scrape, TTL), an toàn đa luồng, wire protocol, tầng TCP `Transport`, thư viện client tracker có backoff, đo tải tracker
Ngô Minh Tùng B23DCDT279	B – Peer core	B1–B7: metainfo, lưu trữ và kiểm hash chunk, rarest-first + endgame, tải song song, upload có giới hạn băng thông, chịu lỗi khi mất nguồn, CLI `peer`
Tạ Quang Vinh B23DCDT289	C – Mô phỏng & đánh giá	C1–C7: bộ khởi chạy kịch bản, engine churn, giả lập mạng, thu số liệu, kiểm thử E2E, thí nghiệm và biểu đồ, demo trực tiếp

Phần việc chung (báo cáo, slide, README, review chéo) được chia đều: ai phụ trách module nào thì viết phần báo cáo và trình bày phần đó; review code chéo theo vòng tròn A→B→C→A.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 Mô hình mạng ngang hàng (P2P)

Trong mạng P2P, các nút (peer) có vai trò ngang nhau: mỗi nút vừa là client vừa là server. Một số đặc điểm quan trọng:

- **Khả năng mở rộng:** tài nguyên phục vụ tăng theo số nút tham gia.
- **Không có điểm lỗi duy nhất** về dữ liệu: một nút sập thì dữ liệu vẫn còn ở nút khác (nếu đã được nhân bản).
- **Thách thức:** các nút không tin cậy, vào ra liên tục (churn), dữ liệu có thể bị sửa, mạng không đồng nhất.

Mạng P2P chia làm hai loại chính: *có cấu trúc* (DHT như Chord, Kademlia) và *không cấu trúc*. BitTorrent cổ điển là dạng lai: dùng một tracker tập trung để **tìm peer**, còn việc **truyền dữ liệu** hoàn toàn ngang hàng.

2.2 Giao thức BitTorrent

2.2.1 Các thành phần

- **Metainfo (file .torrent):** chứa tên file, kích thước, kích thước chunk (`piece_length`), danh sách hash SHA-1 của từng chunk và địa chỉ tracker. Hash của phần `info` gọi là `info_hash` – định danh duy nhất của torrent.
- **Tracker:** máy chủ HTTP ghi nhận peer nào đang tham gia swarm nào. Peer gửi `announce` định kỳ, tracker trả về danh sách một số peer ngẫu nhiên.
- **Swarm:** tập các peer cùng chia sẻ một torrent, gồm *seed* (đủ file) và *leecher* (đang tải).
- **Wire protocol:** giao thức TCP giữa hai peer để trao đổi bitfield, yêu cầu và gửi chunk.

2.2.2 Chunk và kiểm tra tính toàn vẹn

File được chia thành các chunk có kích thước cố định (chunk cuối có thể ngắn hơn). Khi nhận đủ một chunk, peer tính SHA-1 và so với hash trong metainfo; nếu sai thì loại bỏ và tải lại từ

nguồn khác. Nhờ vậy peer không cần tin vào bất kỳ nguồn nào – chỉ cần tin vào file metainfo.

2.2.3 Chiến lược chọn chunk: rarest-first và endgame

Nếu mọi peer đều tải chunk theo thứ tự 0, 1, 2,…thì các chunk cuối sẽ rất hiếm, và khi seed rời đi thì swarm có thể mất hẳn những chunk đó. **Rarest-first** chọn chunk mà ít peer có nhất trước, giúp các bản sao được phân tán đều và tăng độ sẵn sàng của dữ liệu. Khi gần xong, peer vào chế độ **endgame**: gửi yêu cầu cùng một chunk tới nhiều peer để một nguồn chậm không làm "treo" phần đuôi.

2.2.4 Choke / unchoke

Mỗi peer chỉ phục vụ đồng thời một số lượng giới hạn peer khác (các peer được *unchoke*); số còn lại bị *choke*. Cơ chế này giúp chia băng thông upload hợp lý và tránh quá tải.

2.3 Churn và các mô hình xác suất

Churn là hiện tượng peer liên tục tham gia và rời mạng. Để mô phỏng thực tế, nhóm dùng các mô hình phổ biến trong nghiên cứu P2P:

- **Thời điểm đến theo quá trình Poisson** với tốc độ λ: khoảng cách giữa hai lần đến tuân theo phân phối mũ $T \sim \text{Exp}(\lambda)$, trung bình $1/\lambda$ giây.
- **Thời gian sống theo phân phối mũ hoặc Pareto.** Pareto có "đuôi nặng": nhiều peer sống rất ngắn, một số ít sống rất lâu – giống hành vi người dùng thật. Với tham số $\alpha > 1$ và trung bình μ, ta chọn $x_m = \mu(\alpha - 1)/\alpha$.
- **Rời đàng hoàng** (gửi `stopped` cho tracker) và **sập** (bị kill, không kịp báo ai) – hai trường hợp này cần được hệ thống phát hiện theo cách khác nhau.
- **Flash crowd**: rất nhiều peer ập vào trong thời gian ngắn, thường gặp khi một file vừa được phát hành.

2.4 Các khái niệm hệ phân tán liên quan

Bảng 2.1: Liên hệ giữa đề tài và các khái niệm của môn học

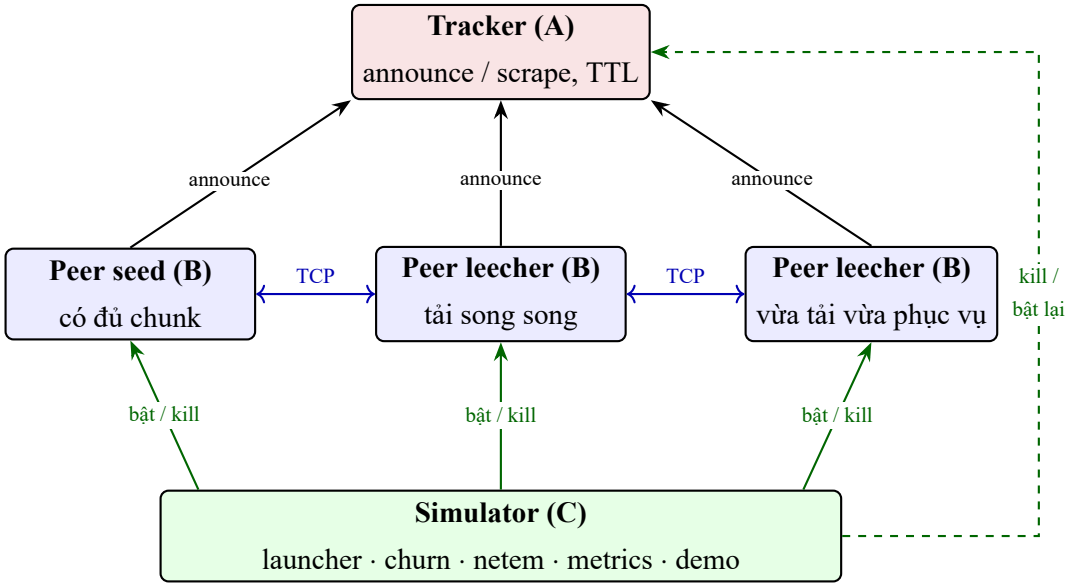
Khái niệm	**Thể hiện trong hệ thống**
Nhân bản (replication)	Mỗi chunk tải xong lập tức được phục vụ lại, số bản sao tăng dần theo thời gian
Phát hiện lỗi	Tracker xóa peer quá TTL; peer phát hiện mất kết nối qua timeout/keepalive
Chịu lỗi & phục hồi	Đổi nguồn khi mất peer; tracker không lưu trạng thái, swarm tự dựng lại từ announce
Đồng thời	Tracker xử lý nhiều request bằng đa luồng + khóa; peer tải song song nhiều chunk
Tính nhất quán dữ liệu	Mọi chunk kiểm SHA-1, file cuối kiểm SHA-256 với file gốc
Bảo mật cơ bản	Chunk sai hash bị từ chối, peer gửi sai nhiều lần bị cấm

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

3.1 Kiến trúc tổng thể

Hệ thống gồm ba thành phần chạy như các tiến trình độc lập: tracker, các peer (seed/leecher) và bộ mô phỏng (simulator). Hình 3.1 mô tả cách chúng giao tiếp với nhau.



Hình 3.1: Kiến trúc tổng thể của hệ thống (mỗi cặp peer đều có thể nối TCP trực tiếp)

Ý tưởng chính: **tracker chỉ dùng để tìm peer**, không đi qua dữ liệu. Dữ liệu đi trực tiếp giữa các peer qua TCP. Simulator đứng bên ngoài: bật/tắt tiến trình, đọc log JSON từ stdout của chúng để đo đạc. Vì thế khi tracker sập, các peer đã biết nhau vẫn tiếp tục trao đổi bình thường.

3.2 Năm hợp đồng giao tiếp

Để ba người làm song song mà không chờ nhau, nhóm chốt trước 5 hợp đồng giao tiếp và ghi lại trong thư mục `docs/`. Mọi thay đổi hợp đồng cần cả ba đồng ý.

Bảng 3.1: Các hợp đồng giao tiếp giữa các module

#	Hợp đồng	Người viết	Nội dung
1	Metainfo	B	JSON: `name, length, piece_length, pieces[], info_hash, announce`
2	API tracker	A	`GET /announce`, `GET /scrape`, các event `started/completed/stopped`
3	Wire protocol	A	Handshake 68 byte; khung 4 byte độ dài + 1 byte loại thông điệp
4	CLI của peer	B	`--torrent --port --dir --role --max-up-kbps`, mã thoát
5	Log sự kiện	C	JSON Lines ra stdout + định dạng kịch bản YAML

3.2.1 Hợp đồng 1 – Metainfo

```
{
  "announce": "http://127.0.0.1:8000/announce",
  "info": {
    "name": "sample_data.bin",
    "length": 2097152,
    "piece_length": 131072,
    "pieces": ["a94a8fe5cc...", "109f4b3c50...", "..."]
  },
  "info_hash": "d14a028c2a3a2bc9476102bb288234c415a2b01f"
}
```

`info_hash` là SHA-1 của phần `info` sau khi chuẩn hóa JSON (sắp xếp khóa, không khoảng trắng), nên hai máy tạo metainfo từ cùng một file sẽ ra cùng `info_hash`.

3.2.2 Hợp đồng 2 – API tracker

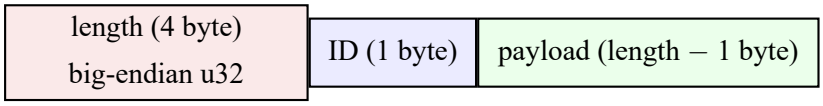
Peer gọi `GET /announce?info_hash=..&peer_id=..&port=..&event=..&left=...` Tracker trả về:

```
{ "interval": 30, "complete": 5, "incomplete": 12,
  "peers": [ {"peer_id": "seed-9001", "ip": "127.0.0.1", "port": 9001} ] }
```

Danh sách tối đa 50 peer ngẫu nhiên, **không chứa chính peer gọi**. `left = 0` nghĩa là seed. Event `stopped` xóa peer ngay; peer không announce lại trong `TTL` giây sẽ bị xóa tự động.

3.2.3 Hợp đồng 3 – Wire protocol

Sau khi kết nối TCP, hai bên trao đổi **handshake** 68 byte: `pstrlen=19`, chuỗi `"BitTorrent protocol"`, 8 byte dự trữ, 20 byte `info_hash`, 20 byte `peer_id`. Bên nhận kiểm tra `info_hash` khớp thì mới trả lời. Các thông điệp sau đó có khung:



Hình 3.2: Khung thông điệp của wire protocol

Bảng 3.2: Các loại thông điệp giữa hai peer

ID	Tên	Payload	Ý nghĩa
–	keepalive	(length = 0)	Giữ kết nối sống
0	choke	–	Không phục vụ yêu cầu của bên kia nữa
1	unchoke	–	Bắt đầu phục vụ
2	interested	–	Muốn tải chunk mà bên kia có
3	not_interested	–	Không cần gì từ bên kia
4	have	`index: u32`	Vừa có thêm chunk `index`
5	bitfield	bitmap	Danh sách chunk đang có (gửi ngay sau handshake)
6	request	`index, begin, length`	Yêu cầu một block trong chunk
7	piece	`index, begin, block`	Gửi dữ liệu block
8	cancel	`index, begin, length`	Hủy yêu cầu (dùng trong endgame)

3.2.4 Hợp đồng 4 – CLI của peer

```
py -m peer --torrent <file.torrent.json> --port 9002 --dir <thư mục>
           --role seed|leech --peer-id leech-9002 --max-up-kbps 256
           [--exit-on-complete]
```

Mã thoát: `0` – thoát bình thường; `1` – tham số hoặc metainfo không hợp lệ; `2` – file của seed không khớp hash; `3` – không thể tải xong (hết nguồn, tracker không phản hồi quá lâu); `130` – leecher bị ngắt bằng SIGINT. Peer bắt `SIGINT`, `SIGTERM` và `SIGBREAK` (Windows) để gửi `stopped` cho tracker trước khi thoát.

3.2.5 Hợp đồng 5 – Log sự kiện và kịch bản

Mỗi tiến trình in ra stdout **một đối tượng JSON trên một dòng**, flush ngay sau mỗi dòng để simulator nhận theo thời gian thực:

```
{"ts": 1762850000.123, "peer_id": "leech-9002", "event": "piece_done",
 "chunk_index": 5, "meta": {"bytes": 131072, "from_peer": "seed-9001"}}
```

Các sự kiện chính:

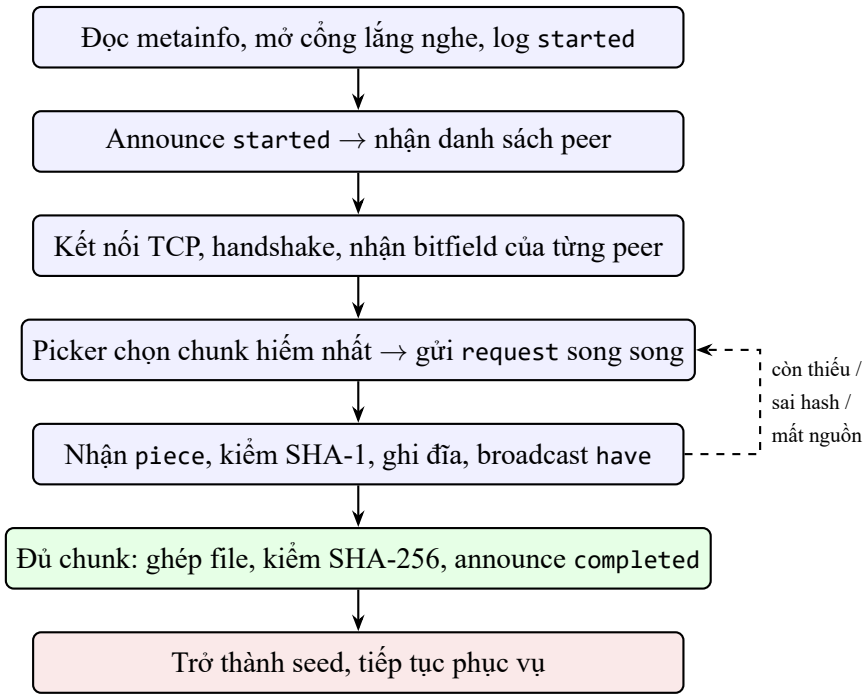
- vòng đời: `started`, `announce_ok`, `completed`, `stopped`;
- truyền chunk: `piece_done`, `piece_uploaded`, `invalid_chunk`;
- lỗi và phục hồi: `peer_lost`, `switch_source`.

Peer bị kill sẽ không ghi được `stopped` – đây chính là cách phân biệt "rời đàng hoàng" với "sập".

Kịch bản thí nghiệm được mô tả bằng file YAML, ví dụ kịch bản *high churn*:

```yaml
swarm:   { num_seeds: 1, num_leechers: 2, base_port: 9001, max_up_kbps: 256 }
torrent: { file_size_bytes: 2097152, piece_length: 131072 }
churn:
  enabled: true
  mode: "poisson"
  arrival_rate_per_sec: 0.7
  mean_lifetime_sec: 8.0
  lifetime_dist: "pareto"
  pareto_alpha: 1.8
  crash_prob: 0.5        # 50% crash, 50% graceful leave
netem:      { latency_ms: 50 }
simulation: { duration_sec: 35, random_seed: 42 }
```

3.3 Luồng hoạt động của một leecher



Hình 3.3: Các bước chính của một leecher

CHƯƠNG 4

CÀI ĐẶT HỆ THỐNG

Toàn bộ mã nguồn viết bằng Python (≥ 3.10), chỉ dùng thêm `PyYAML` và `matplotlib` cho phần mô phỏng. Cấu trúc thư mục:

```
src/tracker/        A1-A3  máy chủ tracker, quản lý swarm, TTL
src/protocol/       A4     mã hóa / giải mã thông điệp wire protocol
src/network/        A5     interface Transport + TcpTransport
src/trackerclient/  A6     client announce định kỳ + backoff
src/torrent/        B1     Metainfo + công cụ make-torrent
src/storage/        B2     Bitfield, PieceStorage (kiểm hash)
src/piece_picker/   B3     rarest-first + endgame
src/peer/           B4-B7  downloader, uploader, session, CLI peer
src/sim/            C1-C7  launcher, churn, netem, metrics, report
tests/              unit test + tests/e2e/run_all.py
scenarios/*.yaml    các kịch bản thí nghiệm
demo/run_demo.py    demo một lệnh
```

4.1 Module A – Tracker và tầng mạng

4.1.1 Máy chủ tracker (A1–A3)

Tracker dùng `ThreadingHTTPServer` của thư viện chuẩn: mỗi request được xử lý trên một luồng riêng. Trạng thái nằm trong lớp `SwarmManager`, là một từ điển hai tầng `{info_hash: {peer_id: PeerInfo}}`. Mọi thao tác đọc/ghi đều được bảo vệ bằng **một** `threading.Lock` nên không có race condition khi nhiều peer announce cùng lúc.

```python
@dataclass
class PeerInfo:
    peer_id: str
    ip: str
    port: int
    left: int                    # 0 -> seeder
    last_announce: float = field(default_factory=time.time)
    event: str = "started"
```

Xử lý `announce`:

- `started` / rỗng: thêm hoặc cập nhật peer, làm mới `last_announce`.
- `completed`: đánh dấu peer là seed, tăng bộ đếm `downloaded`.
- `stopped`: xóa peer khỏi swarm ngay lập tức.
- Trả về tối đa 50 peer chọn ngẫu nhiên (`random.sample`), bỏ chính người gọi.

Phát hiện peer chết (A2): một luồng nền chạy mỗi `interval` giây, xóa các peer có `now - last_announce > TTL`. Peer bị kill không gửi được `stopped` nhưng vẫn biến mất khỏi danh sách sau đúng TTL.

Phục hồi sau khi khởi động lại (A3): tracker **không lưu trạng thái ra đĩa**. Khi bị kill rồi bật lại, swarm rỗng; nhưng vì mọi peer announce định kỳ, swarm sẽ tự dựng lại sau tối đa một chu kỳ `interval`. Cách làm này đơn giản mà vẫn đúng, vì thông tin của tracker chỉ là "gợi ý" để tìm peer, không phải dữ liệu quan trọng.

4.1.2 Wire protocol (A4)

Module `protocol` định nghĩa mỗi loại thông điệp là một `dataclass` (`Handshake`, `Bitfield`, `Request`, `Piece`,…) cùng hàm `encode`/`decode`. Đóng gói dùng `struct.pack(">IB", length, msg_id)`. Bộ giải mã kiểm tra độ dài, ID hợp lệ và kích thước payload; dữ liệu hỏng sẽ ném lỗi thay vì làm treo kết nối. Unit test kiểm tra round-trip cho mọi loại thông điệp.

4.1.3 Tầng kết nối TCP (A5)

Interface `Transport` tách logic peer khỏi chi tiết mạng:

```
class Transport(ABC):
    async def start(host, port, info_hash, peer_id, listener) -> None
    async def connect(remote_ip, remote_port) -> PeerConnection
    async def send(conn, message) -> None
    async def disconnect(conn) -> None

class TransportListener(ABC):
    async def on_connected(conn) -> None
    async def on_disconnected(conn, reason) -> None
    async def on_message(conn, message) -> None
```

`TcpTransport` cài đặt bằng `asyncio`: có timeout khi kết nối, keepalive định kỳ, giới hạn số kết nối, và gọi `on_disconnected` khi đầu bên kia đóng hoặc mất kết nối. Nhờ interface này, trong lúc A chưa xong, B vẫn có thể thử nghiệm bằng `FakeTransport` chạy trong bộ nhớ.

4.1.4 Client tracker có backoff (A6)

`TrackerClient` announce định kỳ theo `interval` tracker trả về. Khi tracker không phản hồi, client thử lại với **exponential backoff có jitter**:

$$t_{\text{chờ}} = \min\left(t_0 \cdot 2^{k},\ t_{\max}\right) + U(0,\ 0.25 \cdot t_{\text{chờ}})$$

với $t_0 = 2\text{s}$, $t_{\max} = 60\text{s}$, k là số lần thất bại liên tiếp. Jitter giúp các peer không cùng "đổ" vào tracker một lúc ngay khi nó sống lại.

4.2 Module B – Peer core

4.2.1 Metainfo và lưu trữ chunk (B1, B2)

Lớp `Metainfo` đọc/ghi file `*.torrent.json`, kiểm tra hợp lệ (số hash khớp với `length` và `piece_length`, mỗi hash đúng 40 ký tự hex) và tính `info_hash`. Công cụ `make_torrent` sinh metainfo cho file bất kỳ.

`PieceStorage` ghi/đọc chunk theo chỉ số trực tiếp vào file đích bằng `seek`. Trước khi ghi, chunk được kiểm SHA-1; nếu sai thì ném `PieceHashError` và không ghi gì. `Bitfield` là mảng bit đánh dấu chunk đã có, có thể chuyển qua lại với dạng byte để gửi trong thông điệp `bitfield`.

4.2.2 Chọn chunk: rarest-first với endgame (B3)

`PiecePicker` là logic thuần, không có I/O nên dễ kiểm thử. Nó giữ:

- `_availability[i]`: số peer đang có chunk i (cập nhật khi nhận `bitfield`, `have` hoặc khi peer biến mất);
- `_in_flight[i]`: tập peer đang được yêu cầu chunk i.

```
def pick(self, peer, count=1):
    remote = self._peers.get(peer)
    endgame = self.is_endgame()
    candidates = []
    for i in self.own.missing():
        if not remote.has(i):
```

```
            continue
        holders = self._in_flight.get(i)
        if holders is None:
            candidates.append(i)
        elif endgame and peer not in holders:
            candidates.append(i)          # endgame: allow duplicates
    self._rng.shuffle(candidates)         # random tie-break
    candidates.sort(key=lambda i: (self._availability[i],
                                   len(self._in_flight.get(i, ()))))
    chosen = candidates[:count]
    for i in chosen:
        self._in_flight.setdefault(i, set()).add(peer)
    return chosen
```

Xáo trộn trước khi sắp xếp (sắp xếp của Python là ổn định) giúp các chunk có cùng độ hiếm được chọn ngẫu nhiên, tránh trường hợp mọi leecher cùng đuổi theo một chunk. Endgame bật khi mọi chunk còn thiếu đều đang được tải; khi một chunk xong, `piece_done` trả về danh sách peer có yêu cầu trùng để gửi `cancel`.

4.2.3 Tải song song (B4)

`Downloader` quản lý danh sách `RemotePeer` (bitfield, trạng thái bị cấm, thời điểm được thử lại…). Peer chạy một nhóm worker, mỗi worker lấy một cặp (chunk, nguồn) từ picker và tải. Mặc định tối đa **4 chunk tải đồng thời** và **không quá 2 yêu cầu cho mỗi nguồn**, nên khi có nhiều seed thì tải được trải ra nhiều nguồn cùng lúc. Mỗi chunk được chia thành các block 16 KiB gửi bằng `request`.

4.2.4 Phục vụ chunk – upload (B5)

Phía server của peer nhận kết nối, gửi bitfield và trả `piece` cho các `request`. Số peer được unchoke cùng lúc bị giới hạn (choke slot); peer vượt slot nhận `choke` và thử lại sau một khoảng ngẫu nhiên ngắn. Băng thông upload bị giới hạn bằng **token bucket** dùng chung cho mọi kết nối:

```
class TokenBucket:
    def consume(self, nbytes):
        with self._lock:
            now = time.monotonic()
            start = max(now, self._next_free)
            self._next_free = start + nbytes / self.rate
```

```
            wait = self._next_free - now
        if wait > 0:
            time.sleep(wait)
```

Điểm quan trọng: **leecher phục vụ chunk ngay khi vừa có**, không cần chờ tải xong toàn bộ file. Mỗi chunk mới được thông báo cho mọi kết nối bằng `have`.

4.2.5 Chịu lỗi phía peer (B6)

- **Mất nguồn giữa chừng:** khi kết nối đứt hoặc timeout, peer log `peer_lost`, trả chunk đang tải về picker (`peer_gone`) và tải lại từ nguồn khác (log `switch_source`). Nguồn lỗi bị tạm "nghỉ" `PEER_RETRY_SEC = 2` giây trước khi thử kết nối lại.
- **Chunk sai hash:** chunk bị loại, log `invalid_chunk`, nguồn đó bị đánh dấu "xấu" với chunk này; gửi sai **từ 2 lần trở lên** thì nguồn bị cấm hẳn và đóng kết nối.
- **Thiếu nguồn:** khi không còn peer nào có chunk cần, peer announce lại sớm (mỗi 2 giây thay vì chờ `interval`) để xin thêm peer. Nếu 90 giây liền không có nguồn dùng được thì thoát với mã 3.
- **Tracker sập:** vòng announce dùng backoff, bắt đầu 0.5 giây và nhân đôi tới tối đa `interval`; trong lúc đó việc trao đổi chunk với các peer đã biết vẫn tiếp tục.

4.2.6 Tiến trình peer hoàn chỉnh (B7)

`peer.py` ghép tất cả lại: đọc tham số CLI (Hợp đồng 4), ghi log JSON Lines (Hợp đồng 5), cài signal handler để rời đàng hoàng. Peer chạy nhiều luồng: luồng server, luồng announce, luồng quản lý session, các worker tải và luồng broadcast `have`.

4.3 Module C – Mô phỏng và đánh giá

4.3.1 Bộ khởi chạy kịch bản (C1)

`launcher.py` đọc file YAML, sinh file dữ liệu tất định và metainfo, bật tracker, bật các seed/leecher dưới dạng tiến trình con, gom stdout của chúng vào `logs/<kịch bản>/events.jsonl`. Khi kết thúc, launcher dừng đàng hoàng mọi tiến trình (trên Windows dùng `CTRL_BREAK_EVENT` kèm file `.sim_stop`); tiến trình không thoát sau 4 giây sẽ bị kill để không còn tiến trình mồ côi.

4.3.2 Engine churn (C2)

`ChurnGenerator` sinh lịch các sự kiện `join`, `leave_graceful`, `crash`. Mọi số ngẫu nhiên lấy từ `random.Random(seed)`, và mỗi loại (thời điểm đến, thời gian sống, kiểu rời) dùng một luồng số ngẫu nhiên riêng – nên đổi `crash_prob` không làm lệch thời điểm đến. Cùng `random_seed` luôn cho ra cùng lịch churn; lịch được "ký" bằng SHA-256 (`churn_fingerprint`) để kiểm tra hai lần chạy có giống nhau không.

4.3.3 Giả lập mạng (C3)

`netem.py` giới hạn băng thông upload của từng peer (qua `--max-up-kbps`), thêm độ trễ, kill/bật lại tracker theo lịch, kill peer giữa chừng và cho một peer cố tình gửi chunk hỏng để thử cơ chế kiểm hash.

4.3.4 Thu số liệu, kiểm thử và demo (C4, C5, C7)

- **C4 – metrics:** đọc log JSONL và xuất CSV: thời gian tải từng peer, thông lượng theo giây, số bản sao của từng chunk theo thời gian, tỉ lệ hoàn tất, số announce tới tracker mỗi giây.
- **C5 – kiểm thử E2E:** `tests/e2e/run_all.py` chạy toàn bộ unit test và các kịch bản, so SHA-256 file tải về với file gốc, in bảng PASS/FAIL.
- **C7 – demo:** `demo/run_demo.py` hiển thị dashboard trên terminal: số peer, vai trò, bitfield dạng `[####....] 4/16`, số chunk đã upload, sự kiện gần nhất và danh sách 7 tình huống bắt buộc – mỗi mục chỉ được đánh dấu khi sự kiện thật xảy ra.

Ngoài ra, nhóm C viết `StubPeer` và `StubTracker` – bản tối giản nhưng chạy thật (TCP, SHA-1, rarest-first) – để làm mô phỏng ngay từ đầu khi peer và tracker thật chưa xong. Khi module thật hoàn thành, chỉ cần đổi tham số `--peer-script "-m peer"` và `--tracker-script "-m src.tracker"`, không cần sửa mã của C.

CHƯƠNG 5

KIỂM THỬ VÀ THỰC NGHIỆM

5.1 Môi trường thực nghiệm

- Hệ điều hành Windows 11 và Linux, Python 3.13 / 3.14, mọi tiến trình chạy trên `127.0.0.1`.
- File dữ liệu 2 MiB sinh tất định, chia 16 chunk × 128 KiB.
- Upload mỗi peer giới hạn 256 KiB/s (trừ khi kịch bản ghi khác), độ trễ thêm 50 ms ở một số kịch bản.
- Churn sinh với `random_seed = 42`.

5.2 Kiểm thử

Bảng 5.1: Các nhóm kiểm thử

Nhóm	Nội dung
Unit test	Round-trip mọi thông điệp wire; metainfo tạo/đọc/kiểm tra; storage từ chối chunk sai hash; picker chọn đúng chunk hiếm; tracker TTL, `stopped`; client tracker backoff; churn tất định theo seed
Functional	Đúng hợp đồng CLI và mã thoát; file tải về khớp SHA-256 và từng chunk khớp SHA-1
Concurrency	High churn, flash crowd – nhiều peer cùng tải, cùng announce
Fault & Recovery	Kill nguồn giữa chừng → `peer_lost` → `switch_source`; rời đàng hoàng vs sập; tracker sập rồi bật lại
Security	Peer gửi chunk hỏng bị phát hiện, chunk bị từ chối, peer bị cấm
Performance	Giới hạn băng thông đúng mức cấu hình; multi-seed nhanh hơn baseline

Chạy toàn bộ bằng một lệnh `py tests/e2e/run_all.py` (15 hạng mục, gồm 59 unit test và 12 kiểm thử E2E, khoảng 2–3 phút); tất cả đều PASS.

5.3 Các kịch bản thí nghiệm

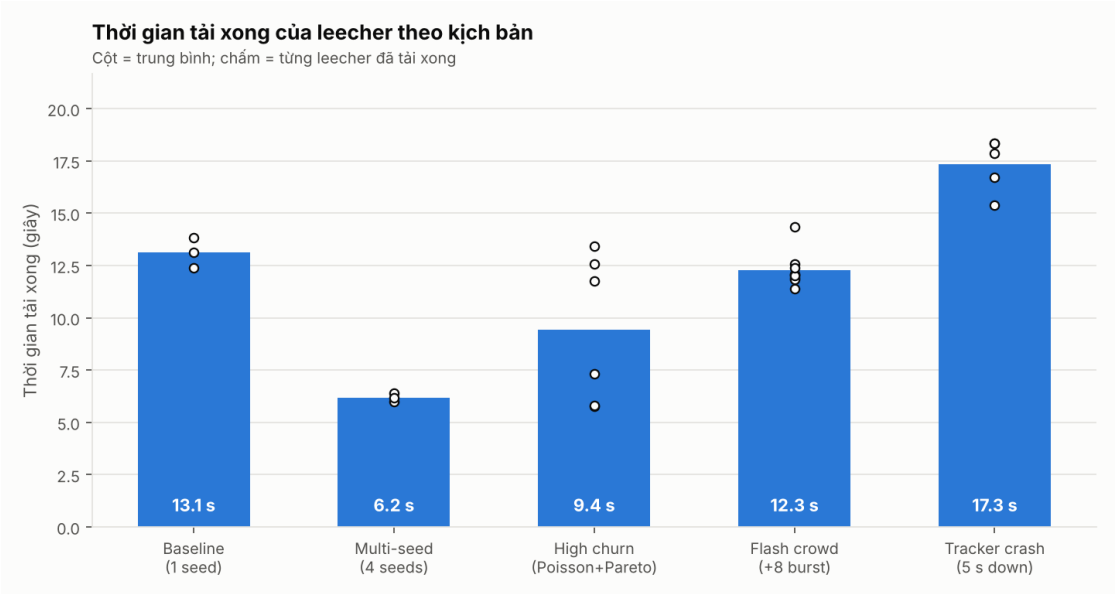
Bảng 5.2: Các kịch bản thí nghiệm

Kịch bản	**Mô tả**
Baseline	1 seed, 3 leecher, mạng ổn định
Multi-seed	4 seed, 3 leecher – kiểm tra hiệu quả tải song song
High churn	1 seed + 2 leecher tĩnh; leecher động đến theo Poisson ($\lambda = 0.7$/s), sống theo Pareto (TB 8 s, $\alpha = 1.8$), 50% rời bằng cách sập
Flash crowd	8 leecher ập vào trong 0.5 s, sau đó rời đồng loạt
Tracker crash	Tracker bị kill ở giây 3, bật lại ở giây 8; 2 leecher đến đúng lúc tracker đang sập

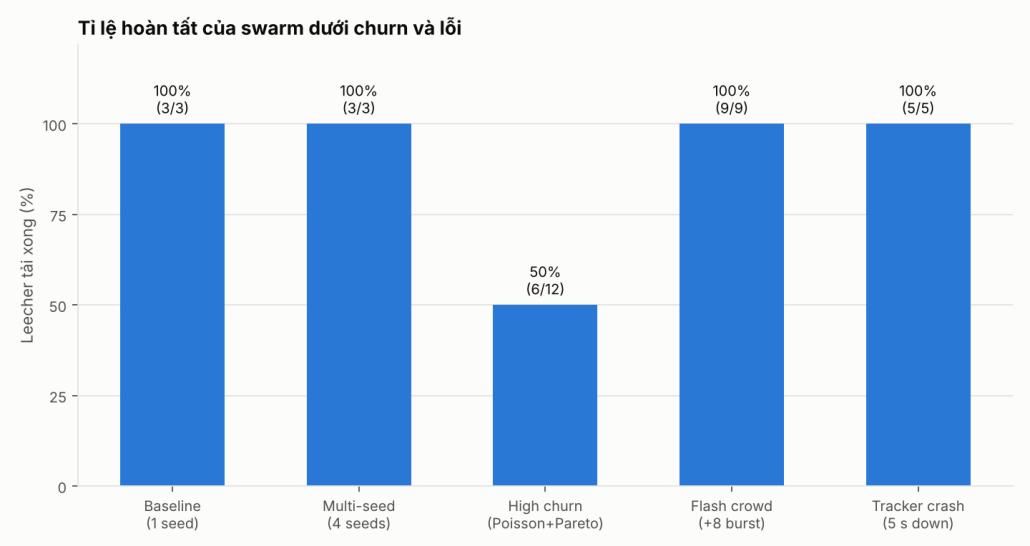
5.4 Kết quả

Bảng 5.3: Tổng hợp kết quả thí nghiệm

Kịch bản	**Seed**	**Leech**	**Hoàn tất**	**Thời gian tải (s)**		**Nhân bản TB**	**Leech upload**
				TB	**min–max**		
Baseline	1	3	100%	13.1	12.4–13.8	2.40	44%
Multi-seed	4	3	100%	6.2	6.0–6.4	5.27	8%
High churn	1	12	50%	9.4	5.8–13.4	3.62	72%
Flash crowd	1	9	100%	12.3	11.4–14.3	4.76	82%
Tracker crash	2	5	100%	17.3	15.4–18.4	4.03	51%



Hình 5.1: Thời gian tải của các leecher theo từng kịch bản



Hình 5.2: Tỉ lệ leecher tải xong theo từng kịch bản

5.4.1 Baseline

Cả 3 leecher đều tải xong, trung bình 13.1 s. Điều đáng chú ý là **44%** dữ liệu được phục vụ bởi chính các leecher chứ không phải seed. Dù seed chỉ có 256 KiB/s, các leecher trao đổi chunk cho nhau ngay khi có nên thông lượng đỉnh của cả swarm lên tới 1280 KiB/s – đây đúng là ưu điểm của P2P.

5.4.2 Multi-seed

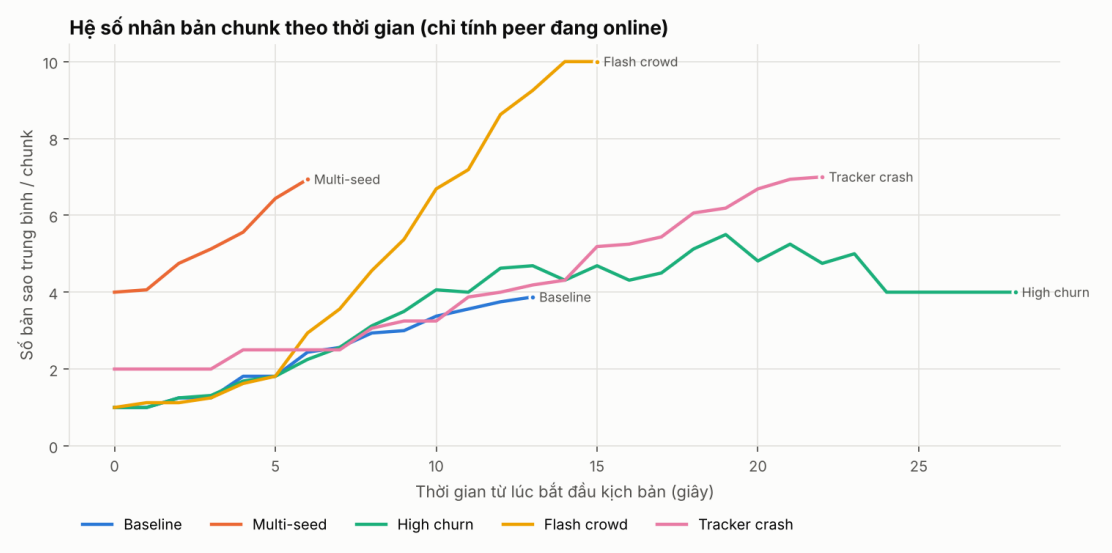
Với 4 seed, thời gian tải giảm còn 6.2 s, **nhanh hơn 2.12 lần** so với baseline. Tốc độ không tăng đủ 4 lần vì ở baseline các leecher đã bù một phần băng thông, và mỗi chunk vẫn có một độ trễ cố định không giảm theo số seed. Tỉ lệ leecher upload giảm còn 8% vì seed đã đủ nhanh.

5.4.3 High churn

Có 12 leecher tham gia, trong đó nhiều peer bị kill đột ngột. Hệ thống ghi nhận **20 lần** `peer_lost` và **8 lần** `switch_source`: peer phát hiện nguồn chết và tự chuyển sang nguồn khác, các leecher tĩnh vẫn tải xong đúng hash. Tỉ lệ hoàn tất chỉ đạt 50% vì phân phối Pareto có nhiều peer sống rất ngắn, rời mạng trước khi kịp tải đủ – đây là hành vi mong đợi, không phải lỗi. Do seed gốc luôn online nên mọi chunk luôn có ít nhất 1 bản sao.

5.4.4 Flash crowd

8 leecher ập vào cùng lúc nhưng vẫn hoàn tất 100%, trung bình 12.3 s. Leecher phục vụ tới **82%** dữ liệu: một seed không đủ băng thông cho cả đám đông, nhưng rarest-first giúp các leecher giữ những chunk khác nhau nên có thể trao đổi cho nhau. Tracker chịu tải đỉnh 13 announce/giây.

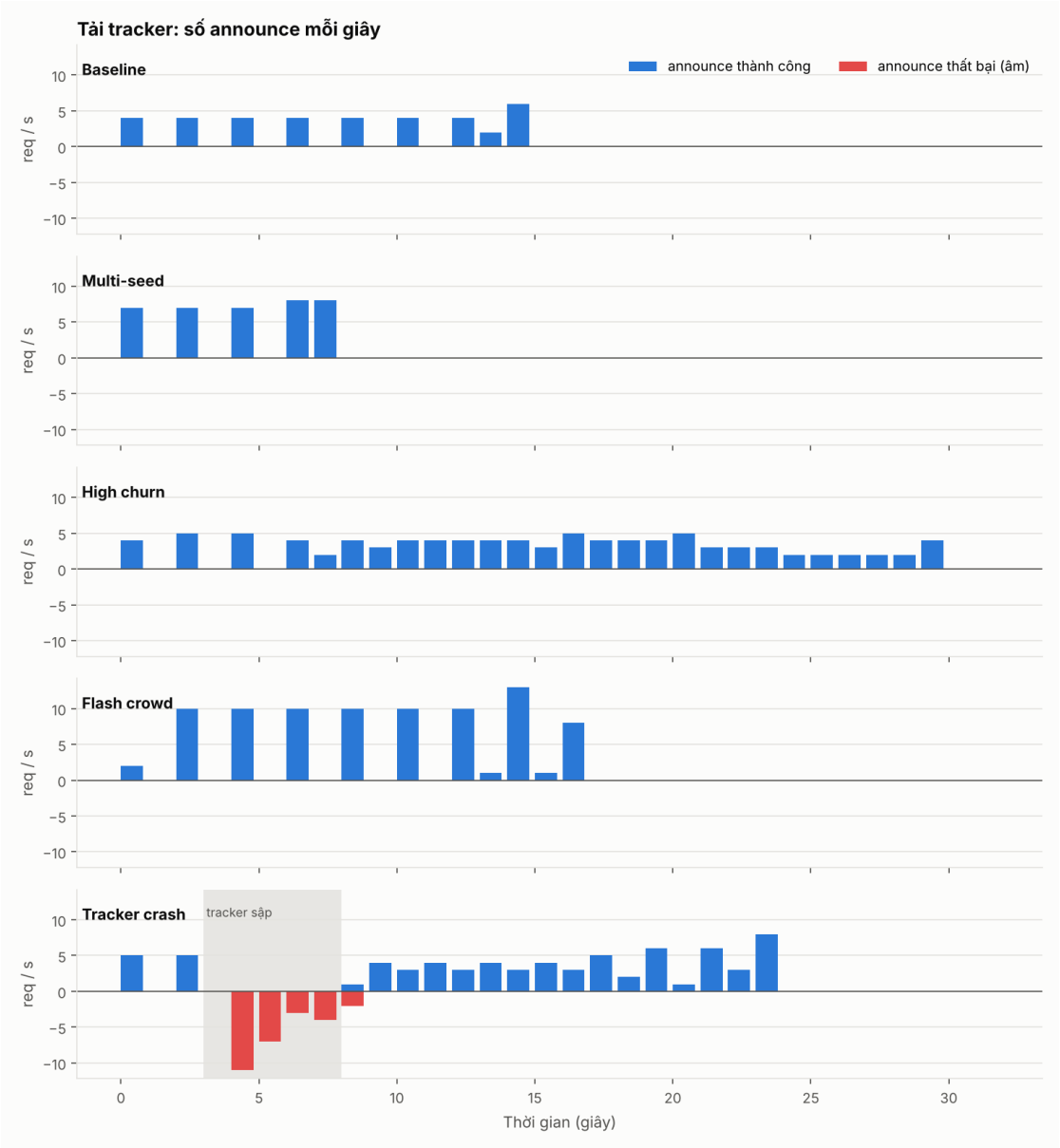


Hình 5.3: Hệ số nhân bản trung bình của chunk theo thời gian

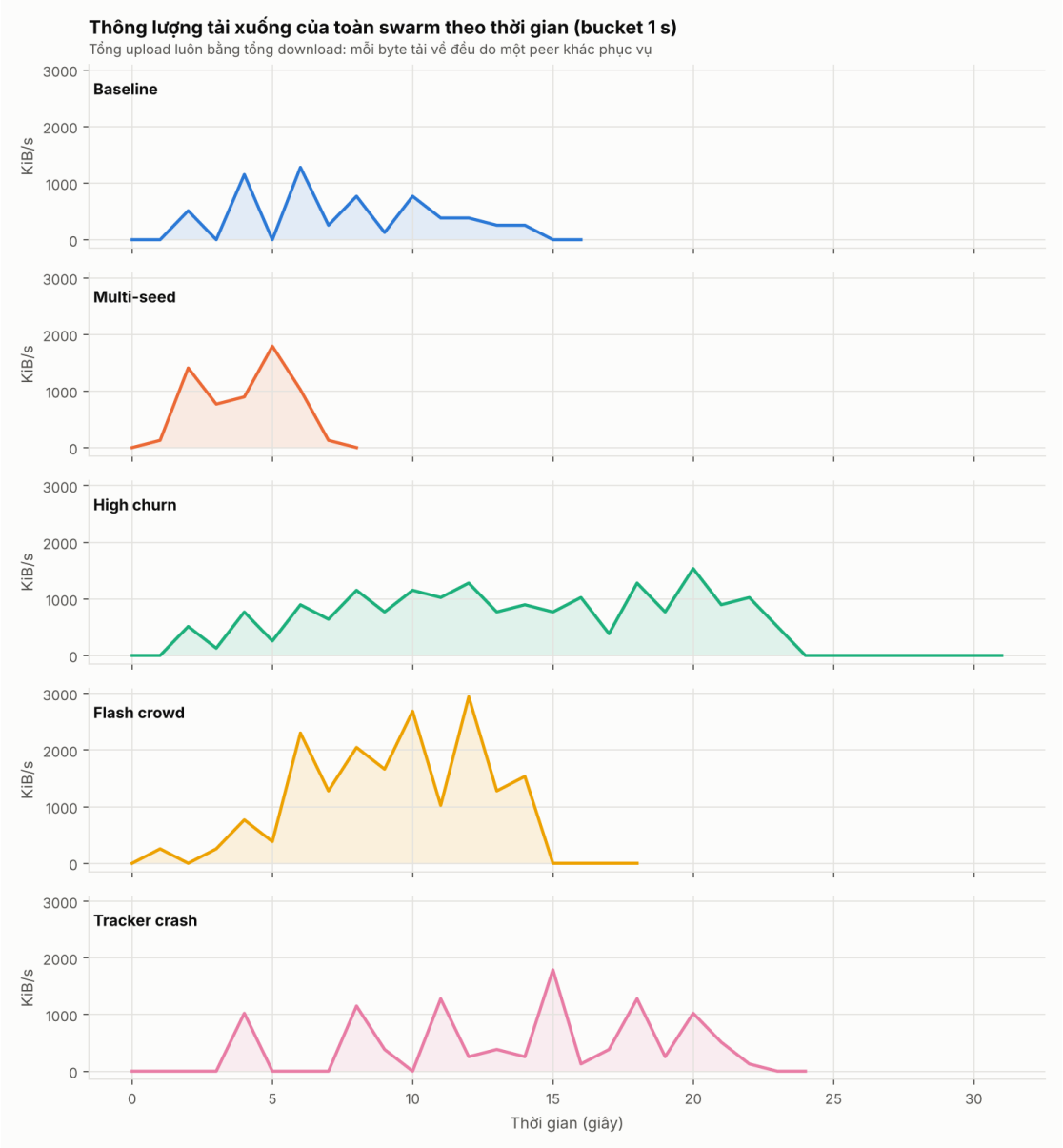
Hình 5.3 cho thấy số bản sao của chunk tăng nhanh sau đợt ập vào và sụt mạnh khi các peer rời đồng loạt. Seed gốc là điều kiện để không chunk nào biến mất hoàn toàn.

5.4.5 Tracker crash

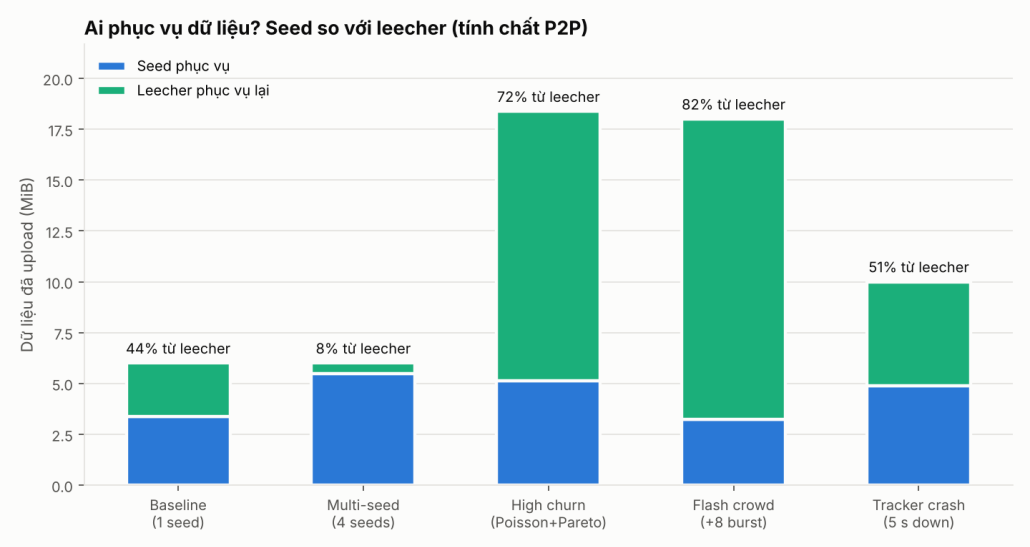
Tracker ngừng hoạt động khoảng 5.1 s, trong thời gian đó có 27 lần announce thất bại. Tuy vậy, tỉ lệ hoàn tất vẫn là **100%**: các peer đã biết nhau vẫn trao đổi chunk trực tiếp. Hai leecher đến lúc tracker đang sập không tìm được nguồn cho tới khi tracker sống lại (nhờ announce thử lại với backoff). Sau khi tracker khởi động lại, swarm được đăng ký lại đầy đủ sau khoảng 1.5 s mà tracker không cần lưu bất kỳ trạng thái nào.



Hình 5.4: Số announce tới tracker mỗi giây (đánh dấu khoảng tracker sập)



Hình 5.5: Thông lượng tải xuống / tải lên của swarm theo thời gian



Hình 5.6: Tỉ lệ dữ liệu do seed và do leecher phục vụ

5.5 Đo tải tracker

Nhóm dùng script `scripts/benchmark_tracker.py` gửi 500 request `announce` với 20 luồng song song tới tracker thật trên loopback:

Bảng 5.4: Kết quả đo tải tracker

Chỉ số	Giá trị
Tổng số request	500
Số luồng đồng thời	20
Thông lượng	589.0 request/s
Tỉ lệ thành công	100% (0 lỗi)
Độ trễ trung bình	25.54 ms
Độ trễ p50 / p95	4.02 ms / 10.84 ms
Độ trễ p99 / max	667.69 ms / 672.94 ms

Phần lớn request được xử lý dưới 11 ms. Một số ít request có độ trễ cao (p99) do hàng đợi kết nối của socket bị đầy trong đợt đầu, khi 20 luồng cùng mở kết nối mới. Với quy mô vài chục peer announce mỗi 2 giây như trong thí nghiệm thì tracker dư sức đáp ứng.

5.6 Demo

Lệnh `py demo/run_demo.py` chạy khoảng 30 giây và kiểm tra đủ 7 tình huống bắt buộc của đề: (1) tạo metadata và chia sẻ; (2) tải song song từ ít nhất 2 peer; (3) leecher upload trước khi tải

xong; (4) dừng một nguồn giữa chừng; (5) tự chuyển nguồn; (6) phát hiện và từ chối chunk sai hash; (7) ghép file và xác minh hash. Kết quả: `RESULT: 7/7`.

CHƯƠNG 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết quả đạt được

- Xây dựng hoàn chỉnh hệ thống chia sẻ file P2P kiểu BitTorrent gồm tracker, peer và bộ mô phỏng; các thành phần chạy như tiến trình riêng và giao tiếp qua mạng thật.
- **Tính đúng:** trong mọi kịch bản, leecher báo `completed` đều có file trùng SHA-256 với file gốc; chunk sai hash luôn bị từ chối.
- **Hiệu năng:** tăng khi thêm nguồn (multi-seed nhanh hơn 2.12 lần) và nhờ leecher phục vụ lại (tới 82% dữ liệu trong flash crowd).
- **Chịu lỗi:** mất nguồn thì tự đổi nguồn; tracker sập không làm gián đoạn việc trao đổi chunk; swarm tự dựng lại sau khi tracker khởi động lại.
- Có bộ kiểm thử tự động, pipeline thí nghiệm sinh CSV/biểu đồ và demo một lệnh.
- Về làm việc nhóm: cách "chốt hợp đồng trước, mỗi người tự viết mock" giúp ba người làm song song gần như không phải chờ nhau, và việc tích hợp cuối cùng chỉ là đổi tham số dòng lệnh.

6.2 Hạn chế

- Mọi thứ chạy trên một máy (loopback) nên độ trễ và băng thông là giả lập, chưa phản ánh hết mạng thật (mất gói, NAT, tường lửa).
- Tracker vẫn là thành phần tập trung: peer mới không thể tham gia khi tracker sập.
- Chỉ giới hạn băng thông chiều upload; cơ chế choke còn đơn giản (chưa có tit-for-tat và optimistic unchoke như BitTorrent gốc).
- Metainfo dùng JSON thay vì bencode nên không tương thích với client BitTorrent thật.
- Kết quả thời gian có dao động nhỏ giữa các lần chạy do lập lịch luồng của hệ điều hành (lịch churn thì tất định).

6.3 Hướng phát triển

- Thêm DHT (Kademlia) để tìm peer không cần tracker, hoặc hỗ trợ nhiều tracker dự phòng.
- Cài đặt choking theo tit-for-tat và optimistic unchoke để khuyến khích chia sẻ công bằng.
- Chạy thử trên nhiều máy thật hoặc dùng `tc netem` / Docker để giả lập mạng chính xác hơn.
- Hỗ trợ torrent nhiều file, định dạng bencode và magnet link để tương thích với client thật.
- Mã hóa kết nối giữa các peer.

TÀI LIỆU THAM KHẢO

[1] A. S. Tanenbaum, M. van Steen, *Distributed Systems*, 4th ed., distributed-systems.net, 2023.

[2] B. Cohen, "The BitTorrent Protocol Specification (BEP 3)", bittorrent.org, 2008.

[3] B. Cohen, "Incentives Build Robustness in BitTorrent", *Workshop on Economics of Peer-to-Peer Systems*, 2003.

[4] A. Legout, G. Urvoy-Keller, P. Michiardi, "Rarest First and Choke Algorithms Are Enough", *ACM IMC*, 2006.

[5] D. Stutzbach, R. Rejaie, "Understanding Churn in Peer-to-Peer Networks", *ACM IMC*, 2006.

[6] P. Maymounkov, D. Mazières, "Kademlia: A Peer-to-Peer Information System Based on the XOR Metric", *IPTPS*, 2002.

[7] Python Software Foundation, *Python 3 Documentation* – `socket`, `asyncio`, `threading`, `http.server`, docs.python.org.